

AI Native Video Workspace

Claude Code — Build Brief operativo

Obiettivo: trasformare l'idea in un prodotto SaaS reale, costruito per vertical slice e pronto a essere sviluppato con Claude Code.

Strategia MVP: Auth → Room → Video → Whiteboard → AI → Gesture → Integration.

1. Visione e posizionamento

Workspace collaborativo video-first con strumenti nativi di collaborazione e AI. Il valore non è una semplice video-call: la stanza diventa un ambiente di lavoro nel quale video, canvas/whiteboard, AI e interazioni gestuali convivono nello stesso flusso.

Principio di prodotto: ridurre il numero di strumenti separati necessari durante una sessione creativa, educativa o di collaborazione remota.

2. MVP: cosa costruire

Must-have: autenticazione; creazione/join room; audio/video realtime; gestione partecipanti; whiteboard/canvas condiviso; AI assistant/gateway; generazione immagini AI con consumo tracciato; salvataggio stato della room; gestione base di errori e permessi.

Secondo livello: hand tracking e gesture engine; integrazioni; analytics più avanzate; billing completo e team features.

Non bloccare l'MVP su funzioni avanzate di computer vision, automazioni complesse o mobile nativo.

3. Architettura tecnica

Frontend: Next.js + TypeScript, React, Tailwind/shadcn-style component system.

Realtime: WebRTC tramite LiveKit o architettura equivalente, con abstraction layer per evitare lock-in.

Backend/data: Supabase/Postgres, Auth, Row Level Security, storage dove appropriato.

Deployment: Vercel per web/app e servizi edge/serverless dove convenienti.

Object storage: Cloudflare R2 per asset e file quando economicamente più efficiente.

Payments: Stripe Billing.

AI: AI Gateway/provider abstraction con logging di provider, modello, token/crediti, latenza e costo stimato.

Computer vision: MediaPipe/Web APIs dove sufficienti; elaborazione client-side quando possibile per privacy e costi.

4. Modello dati iniziale

Entità principali: users, profiles, workspaces, workspace_members, rooms, room_members, room_events, whiteboard_documents, whiteboard_operations/snapshots, ai_requests, ai_usage, assets, subscriptions, invoices/entitlements.

Ogni entità deve avere ownership, timestamps e audit fields. Le autorizzazioni devono essere definite a livello di workspace/room e non affidate solamente al frontend.

5. Realtime e whiteboard

Separare stato persistente e stato effimero. Presence, cursori e segnali realtime non devono trasformarsi automaticamente in scritture DB.

Per il whiteboard usare operazioni/eventi condivisi e snapshot periodici. Prevedere undo/redo, reconnect e conflitti di base.

Per il video: join/leave, mute/unmute, camera state, network state e reconnect devono essere osservabili e testabili.

6. AI gateway

Creare un'interfaccia unica interna, ad esempio AIService, con adapter per ciascun provider.

Ogni richiesta deve produrre: user/workspace/room, provider, model, operation type, input/output metadata, latency, success/failure e costo stimato.

Le funzioni a costo variabile devono avere quota/credit system. L'utente non deve poter generare costi illimitati accidentalmente.

7. Gesture / hand tracking

Implementare dopo il core collaboration loop. Pipeline: camera stream → hand landmarks → gesture classifier → debouncing/cooldown → UI command.

Esempi: open palm = tool/menu, pinch = select/drag, swipe = navigation. Le gesture devono essere configurabili e disattivabili.

Preferire elaborazione locale del video per il riconoscimento delle mani, evitando upload del feed quando non necessario.

8. Sicurezza e GDPR

Minimizzazione dati; consenso e informative coerenti con il prodotto; retention configurabile; cancellazione account/dati; access control server-side; RLS; secrets esclusivamente lato server; rate limiting; logging senza dati sensibili non necessari.

Video e audio non devono essere registrati di default. Se in futuro si introduce recording, servono policy, UI di consenso e gestione retention dedicate.

9. Tempistiche

3–4 settimane — aggressive: MVP tecnico dimostrabile, scope rigidissimo, una sola piattaforma web.

5–7 settimane — realistiche: MVP stabile con polish, testing, error handling, onboarding e billing iniziale.

8–10 settimane — polished: qualità superiore, gesture più solide, analytics, integrazioni e preparazione al primo rilascio commerciale.

Settimana 1: repo, CI, auth, DB, room lifecycle. **Settimana 2:** video/realtime + UX room. **Settimana 3:** whiteboard + persistence. **Settimana 4:** AI gateway + usage tracking. **Settimane 5–6:** gesture, billing, QA. **Settimane 7–10:** polish e post-MVP.

10. Costi di sviluppo

Per un build founder-led con Claude Code, il costo cash iniziale può essere mantenuto basso: dominio, hosting, database, realtime, storage e API AI sono le voci principali.

Budget operativo indicativo per prototipo/MVP: **circa €50–€300/mese** in base a traffico e uso AI, escluso il costo del lavoro umano. In fase commerciale il costo cresce con minuti realtime, storage, egress, richieste AI e utenti attivi.

Il vero obiettivo è misurare il costo per workspace attivo e per sessione, non solo il costo totale mensile.

11. Pricing e monetizzazione

Free: ingresso e test del prodotto con limiti severi.

Creator: circa €12–€19/mese, con limiti superiori e crediti AI inclusi.

Pro: circa €29–€49/mese, collaboration e AI più generosi.

Team: circa €79–€149/mese o pricing per seat, con amministrazione e workspace.

Enterprise: prezzo custom, SSO, governance, SLA e maggiore controllo.

Per le funzioni AI costose conviene combinare subscription + crediti/usage, evitando di includere generazione illimitata nei piani economici.

12. Unit economics

Metriche da implementare subito: CAC, ARPU, gross margin, AI cost/user, realtime cost/user, storage/egress cost/user, churn, activation rate, conversion Free→Paid e LTV.

Formula semplificata: **Gross Margin = (Revenue – variable infrastructure/AI costs) / Revenue.**

Obiettivo iniziale: conoscere il costo marginale di ogni funzione prima di aumentare i limiti dei piani.

13. Scenari di ricavo illustrativi

Scenario 1: 100 clienti paganti × €19 = €1.900 MRR.

Scenario 2: 500 clienti paganti × €29 = €14.500 MRR.

Scenario 3: 1.000 clienti paganti × €39 = €39.000 MRR.

Scenario 4: 5.000 clienti paganti × €39 = €195.000 MRR.

ARR teorico = MRR × 12. Questi sono scenari matematici, non previsioni di vendita.

14. Break-even

Se i costi fissi iniziali fossero €500/mese e il margine lordo medio fosse 75%, il revenue necessario per coprire i costi sarebbe circa €667/mese.

Se i costi fissi fossero €2.000/mese con lo stesso margine, il break-even sarebbe circa €2.667/mese.

Il break-even reale deve includere software, professionisti, commercialista, supporto, marketing, tasse e costi di acquisizione.

15. Repository structure

apps/web — Next.js application.

packages/ui — design system.

packages/realtime — LiveKit abstraction.

packages/ai — provider adapters and usage accounting.

packages/whiteboard — canvas/domain logic.

packages/gesture — CV/gesture logic.

packages/db — schema/client/types.

supabase — migrations/policies.

docs — ADRs, architecture and product decisions.

16. Workflow con Claude Code

Claude Code deve lavorare per piccoli task verificabili. Prima leggere repository e documentazione, poi proporre piano, implementare, eseguire test/lint/typecheck, controllare diff e aggiornare documentazione.

Regola: non creare astrazioni premature. Ogni feature deve avere una Definition of Done e una prova manuale riproducibile.

Non modificare segreti, billing o policy di sicurezza senza esplicitare il cambiamento nel piano.

17. Primo prompt per Claude Code

You are the lead engineer for this project. Read the entire repository before changing code. Build the product incrementally as a production-oriented SaaS, not as a demo.

First create/verify:

1. repository structure;
2. architecture document;
3. environment variable contract;
4. database schema and migrations;
5. auth flow;
6. CI/lint/typecheck/test setup.

Then implement the first vertical slice:

AUTH → CREATE ROOM → JOIN ROOM → ROOM UI SHELL.

Requirements:

- TypeScript strict mode.
- Server-side authorization.
- No secrets in client code.
- Clear separation between UI, domain logic, realtime, database and AI.
- Every meaningful change must be tested.
- Keep the implementation minimal and reversible.
- Do not implement advanced AI, gesture recognition or billing until the room lifecycle is stable.

At the end, report:

- files created/changed;
- commands executed;
- tests and their results;
- known limitations;
- next 3 recommended tasks.

Do not silently expand scope.

18. Primi 10 task

- 1) Bootstrap repository and tooling.
- 2) Configure Supabase/Auth.
- 3) Create workspace/room schema and RLS.
- 4) Build dashboard and create-room flow.
- 5) Build join-room flow and room permissions.
- 6) Integrate realtime video/audio.
- 7) Add participant/presence state.
- 8) Add whiteboard canvas and persistence.
- 9) Add AI gateway + usage ledger.
- 10) Add gesture prototype + billing entitlements.

19. MVP Definition of Done

Un utente può registrarsi, creare una room, invitare/unire un altro utente, usare audio/video, modificare una whiteboard condivisa, utilizzare almeno una funzione AI con costo/usage registrato, uscire e rientrare senza perdere lo stato persistente, e ricevere errori gestiti in modo comprensibile.

Devono esistere test per auth/permissions, room lifecycle, persistence, AI usage accounting e almeno smoke test del percorso principale.

20. Roadmap post-MVP

Fase 2: recording opzionale, AI meeting summary, template, export, collaboration avanzata.

Fase 3: gesture system maturo, automazioni, integrazioni calendar/storage, team administration.

Fase 4: enterprise controls, SSO, audit logs, SLA, API pubblica e marketplace/integration ecosystem.

21. Principio economico finale

Il prodotto deve essere progettato per sapere quanto costa ogni azione. Ogni chiamata AI, minuto realtime, asset generato e operazione di storage significativa deve poter essere attribuita a user/workspace/feature. Questo consente di correggere pricing e limiti prima che la crescita renda il problema costoso.